

RISC-V 函数调用约定和Stack使用

发布于 2023-10-11 08:44:31

1.7K0

举报

文章被收录于专栏：[c++与qt学习](#)

关联问题

[换一批](#)

RISC-V函数调用约定中寄存器的使用规则是什么？

在RISC-V架构下，如何确定函数的参数传递顺序？

RISC-V的Stack使用有哪些特殊的指令？

RISC-V 函数调用约定和Stack使用

引言

[MIT 6.S081 2020 操作系统](#)

本文为MIT 6.S081课程第五节重点笔记整理。

RISC-V vs x86

不同的处理器指令集不一样，而汇编语言中都是一条条指令，所以不同处理器对应的汇编语言必然不一样。

如果你使用RISC-V，你不太能将Linux运行在上面。相应的，大多数现代计算机都运行在x86和x86-64处理器上。x86拥有一套不同的指令集，看起来与RISC-V非常相似。通常你们的个人电脑上运行的处理器是x86，Intel和AMD的CPU都实现了x86。

RISC-V和x86并没有它们第一眼看起来那么相似。RISC-V中的RISC是精简指令集（Reduced Instruction Set Computer）的意思，而x86通常被称为CISC，复杂指令集（Complex Instruction Set Computer）。这两者之间有一些关键的区别：

- 首先是指令的数量。
 - 实际上，创造RISC-V的一个非常大的初衷就是因为Intel手册中指令数量太多了。
 - x86-64指令介绍由3个文档组成，并且新的指令以每个月3条的速度在增加。
 - 因为x86-64是在1970年代发布的，所以我认为现在有多于15000条指令。
 - RISC-V指令介绍由两个文档组成。
- 除此之外，RISC-V指令也更加简单。在x86-64中，很多指令都做了不止一件事情。这些指令中的每一条都执行了一系列复杂的操作并返回结果。但是RISC-V不会这样做，RISC-V的指令趋向于完成更简单的工作，相应的也消耗更少的CPU执行时间。这其实是设计人员的在底层设计时的取舍。并没有一些非常确定的原因说RISC比CISC更好。它们各自有各自的使用场景。
- 相比x86来说，RISC另一件有意思的事情是它是开源的。这是市场上唯一的一款开源指令集，这意味着任何人都可以为RISC-V开发主板。RISC-V是来自于UC-Berkly的一个研

究项目，之后被大量的公司选中并做了支持，网上有这些公司的名单，许多大公司对于支持一个开源指令集都感兴趣。

在日常生活中，可能已经在完全不知情的情况下使用了精简指令集。比如说ARM也是一个精简指令集，高通的Snapdragon处理器就是基于ARM。如果你使用一个Android手机，那么大概率你的手机运行在精简指令集上。如果你使用IOS，苹果公司也实现某种版本的ARM处理器，这些处理器运行在iPad，iPhone和大多数苹果移动设备上，甚至对于Mac，苹果公司也在尝试向ARM做迁移（注，刚刚发布的Macbook）。所以精简指令集出现在各种各样的地方。如果你想现实世界中找到RISC-V处理器，你可以在一些嵌入式设备中找到。所以RISC-V也是有应用的，当然它可能没有x86那么流行。

在最近几年，由于Intel的指令集是在是太大了，精简指令集的使用越来越多。Intel的指令集之所以这么大，是因为Intel对于向后兼容非常看重。所以一个现代的Intel处理器还可以运行30/40年前的指令。Intel并没有下线任何指令。而RISC-V提出的更晚，所以不存在历史包袱的问题。

如果查看RISC-V的文档，可以发现RISC-V的特殊之处在于：它区分了Base Integer Instruction Set和Standard Extension Instruction Set。Base Integer Instruction Set包含了所有的常用指令，比如add，mult。除此之外，处理器还可以选择性的支持Standard Extension Instruction Set。例如，一个处理器可以选择支持Standard Extension for Single-Precision Float-Point。这种模式使得RISC-V更容易支持向后兼容。每一个RISC-V处理器可以声明支持了哪些扩展指令集，然后编译器可以根据支持的指令集来编译代码。

看起来使用x86而不是RISC-V的唯一优势就是能得到性能的提升，但是这里的性能是以复杂度和潜在的安全为代价的，我的问题是为什么我们还在使用x86，而不是使用RISC-V处理器？

- 现在整个世界都运行在x86上，如果你突然将处理器转变成RISC-V，那么你就会失去很多重要的软件支持。
- 同时，Intel在它的处理器里面做了一些有意思的事情，例如安全相关的enclave，这是Intel最近加到处理器中来提升安全性的功能。
- 此外，Intel还实现了一些非常具体的指令，这些指令可以非常高效的进行一些特定的运算。所以Intel有非常多的指令，通常来说对于一个场景都会有一个完美的指令，它的执行效率要高于RISC-V中的同等指令。
- 但是这个问题更实际的答案是，RISC-V相对来说更新一些，目前还没有人基于RISC-V来制造个人计算机，SiFive也就是最近才成为第一批将RISC-V应用到个人计算机的公司。所以，从实际的角度来说，因为不能在RISC-V上运行所有为Intel设计的软件，是我对这个问题的最好的答案。

RISC-V寄存器

Register	ABI Name	Description	Saver
x0	zero	Hard-wired zero	—
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5–7	t0–2	Temporaries	Caller
x8	s0/fp	Saved register/frame pointer	Callee
x9	s1	Saved register	Callee
x10–11	a0–1	Function arguments/return values	Caller
x12–17	a2–7	Function arguments	Caller
x18–27	s2–11	Saved registers	Callee
x28–31	t3–6	Temporaries	Caller
f0–7	ft0–7	FP temporaries	Caller
f8–9	fs0–1	FP saved registers	Callee
f10–11	fa0–1	FP arguments/return values	Caller
f12–17	fa2–7	FP arguments	Caller
f18–27	fs2–11	FP saved registers	Callee
f28–31	ft8–11	FP temporaries	Caller

这个表里面是RISC-V寄存器。

- 寄存器是CPU或者处理器上，预先定义的可以用来存储数据的位置。
- 寄存器之所以重要是因为汇编代码并不是在内存上执行，而是在寄存器上执行，也就是说，当我们在做add，sub时，我们是对寄存器进行操作。
- 所以你们通常看到的汇编代码中的模式是，我们通过load将数据存放在寄存器中，这里的数据源可以是来自内存，也可以来自另一个寄存器。
- 之后我们在寄存器上执行一些操作。如果我们对操作的结果关心的话，我们会将操作的结果store在某个地方。这里的目的地可能是内存中的某个地址，也可能是另一个寄存器。这就是通常使用寄存器的方法。

寄存器是用来进行任何运算和数据读取的最快的方式，这就是为什么使用它们很重要，也是为什么我们更喜欢使用寄存器而不是内存。

当我们调用函数时，你可以看到这里**a0 - a7**寄存器。通常我们在谈到寄存器的时候，我们会用它们的**ABI**名字。不仅是因为这样描述更清晰和标准，同时也因为在写汇编代码的时候使用的也是**ABI**名字。

- 第一列中的寄存器名字并不是超级重要，它唯一重要的场景是在**RISC-V**的**Compressed Instruction**中。
- 基本上来说，**RISC-V**中通常的指令是**64bit**，但是在**Compressed Instruction**中指令是**16bit**。
- 在**Compressed Instruction**中我们使用更少的寄存器，也就是**x8 - x15**寄存器。
- 我猜你们可能会有疑问，为什么**s1**寄存器和其他的**s**寄存器是分开的？
- 因为**s1**在**Compressed Instruction**是有效的，而**s2-11**却不是。除了**Compressed Instruction**，寄存器都是通过它们的**ABI**名字来引用。

a0到**a7**寄存器是用来作为函数的参数。如果一个函数有超过**8**个参数，我们就需要用内存了。从这里也可以看出，当可以使用寄存器的时候，我们不会使用内存，我们只在不得不使用内存的场景才使用它。

表中的第**4**列，**Saver**列，当我们在讨论寄存器的时候也非常重要。它有两个可能的值**Caller**，**Callee**。我经常混淆这两个值，因为它们只差一个字母。我发现最简单的记住它们的方法是：

- **Caller Saved**寄存器在函数调用的时候不会保存
- **Callee Saved**寄存器在函数调用的时候会保存

这里的意思是，一个**Caller Saved**寄存器可能被其他函数重写。

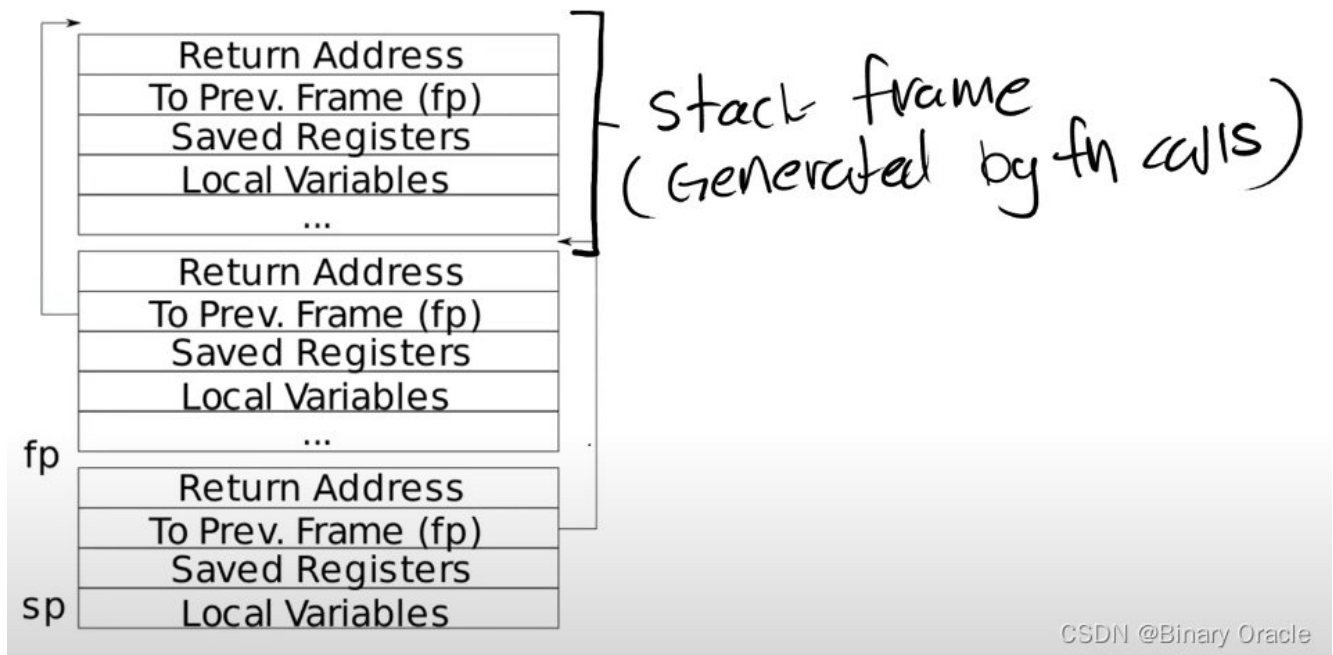
- 假设我们在函数**a**中调用函数**b**，任何被函数**a**使用的并且是**Caller Saved**寄存器，调用函数**b**可能重写这些寄存器。
- 我认为一个比较好的例子就是**Return address**寄存器（注，保存的是函数返回的地址），你可以看到**ra**寄存器是**Caller Saved**，这一点很重要，它导致了当函数**a**调用函数**b**的时候，**b**会重写**Return address**。
- 所以基本上来说，任何一个**Caller Saved**寄存器，作为调用方的函数要小心可能的数据可能的变化；
- 任何一个**Callee Saved**寄存器，作为被调用方的函数要小心寄存器的值不会相应的变化。

如果你们还记得的话，所有的寄存器都是**64bit**，各种各样的数据类型都会被改造的可以放进这**64bit**中。比如说我们有一个**32bit**的整数，取决于整数是不是有符号的，会通过在前面补**32**个**0**或者**1**来使得这个整数变成**64bit**并存在这些寄存器中。

Stack

下面是一个非常简单的栈的结构图，其中每一个区域都是一个**Stack Frame**，每执行一次函数调用就会产生一个**Stack Frame**。

the Stack



CSDN @Binary Oracle

每一次我们调用一个函数，函数都会为自己创建一个Stack Frame，并且只给自己用。函数通过移动Stack Pointer来完成Stack Frame的空间分配。

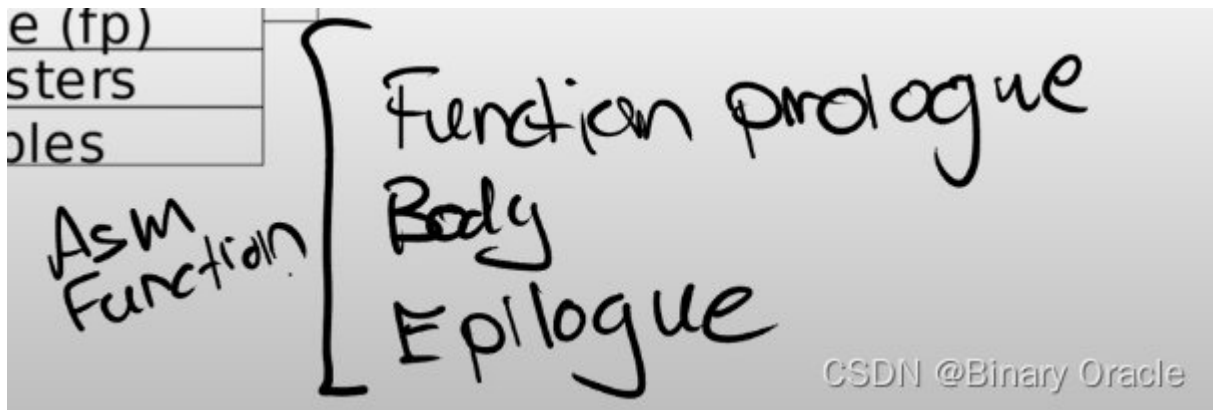
对于Stack来说，是从高地址开始向低地址使用。所以栈总是向下增长。当我们想要创建一个新的Stack Frame的时候，总是对当前的Stack Pointer做减法。一个函数的Stack Frame包含了保存的寄存器，本地变量，并且，如果函数的参数多于8个，额外的参数会出现在Stack中。所以Stack Frame大小并不总是一样，即使在这个图里面看起来是一样大的。不同的函数有不同数量的本地变量，不同的寄存器，所以Stack Frame的大小是不一样的。但是有关Stack Frame有两件事情是确定的：

- Return address总是会出现在Stack Frame的第一位
- 指向前一个Stack Frame的指针也会出现在栈中的固定位置

有关Stack Frame中有两个重要的寄存器，第一个是SP（Stack Pointer），它指向Stack的底部并代表了当前Stack Frame的位置。第二个是FP（Frame Pointer），它指向当前Stack Frame的顶部。因为Return address和指向前一个Stack Frame的的指针都在当前Stack Frame的固定位置，所以可以通过当前的FP寄存器寻址到这两个数据。

我们保存前一个Stack Frame的指针的原因是为了让我们能跳转回去。所以当前函数返回时，我们可以将前一个Frame Pointer存储到FP寄存器中。所以我们使用Frame Pointer来操纵我们的Stack Frames，并确保我们总是指向正确的函数

Stack Frame必须要被汇编代码创建，所以是编译器生成了汇编代码，进而创建了Stack Frame。所以通常，在汇编代码中，函数的最开始你们可以看到Function prologue，之后是函数的本体，最后是Epilogue。这就是一个汇编函数通常的样子。



我们从汇编代码中来看一下这里的操作:

```
sum_to:
    mv t0, a0          # t0 <- a0
    li a0, 0           # a0 <- 0
loop:
    add a0, a0, t0      # a0 <- a0 + t0
    addi t0, t0, -1     # t0 <- t0 - 1
    bnez t0, loop       # if t0 != 0: pc <- loop
    ret
```

CSDN @Binary Oracle

在我们上面的sum_to函数中，只有函数主体，并没有Stack Frame的内容。它这里能正常工作的原因是它足够简单，并且它是一个leaf函数。leaf函数是指不调用别的函数的函数，它的特别之处在于它不用担心保存自己的Return address或者任何其他Caller Saved寄存器，因为它不会调用别的函数。

而另一个函数sum_then_double就不是一个leaf函数了，这里你可以看到它调用了sum_to。

```

.global sum_then_double
sum_then_double:
    addi sp, sp, -16;
    sd ra, 0(sp)
    call sum_to
    li t0, 2
    mul a0, a0, t0
    ld ra, 0(sp)
    addi sp, sp, 16
    ret

```

CSDN @Binary Oracle

所以在这个函数中，需要包含prologue。

```

.global sum_then_double
sum_then_double:
    addi sp, sp, -16;
    sd ra, 0(sp)
    call sum_to
    li t0, 2
    mul a0, a0, t0
    ld ra, 0(sp)
    addi sp, sp, 16
    ret

```

CSDN @Binary Oracle

这里我们对Stack Pointer减16，这样我们为新的Stack Frame创建了16字节的空间。之后我们将Return address保存在Stack Pointer位置。

之后就是调用sum_to并对结果乘以2。最后是Epilogue，

```
.global sum_then_double
sum_then_double:
    addi sp, sp, -16;
    sd ra, 0(sp)
    call sum_to
    li t0, 2
    mul a0, a0, t0
    ld ra, 0(sp)
    addi sp, sp, 16
    ret
```

CSDN @Binary Oracle

这里首先将Return address加载回ra寄存器，通过对Stack Pointer加16来删除刚刚创建的Stack Frame，最后ret从函数中退出。

如果我们删除掉Prologue和Epilogue，然后只剩下函数主体会发生什么？

- sum_then_double将不知道它应该返回的Return address。所以调用sum_to的时候，Return address被覆盖了，最终sum_to函数不能返回到它原本的调用位置。

我们可以看一下具体会发生什么。先在修改过的sum_then_double设置断点，然后执行sum_then_double:


```
Register group: general
zero      0x0      0
ra         0x80006392      0x80006392 <demo2+18>
sp         0x3ffffff9f80   0x3ffffff9f80
gp         0x505050505050505      0x505050505050505
tp         0x0      0x0
t0         0x80002840      2147493952
t1         0x8000000000087fff      -9223372036854218753
t2         0x505050505050505      361700864190383365
fp         0x3ffffff9f90   0x3ffffff9f90
s1         0x80012048      2147557448
a0         0x5      5
a1         0x3ffffff9f9c      274877882268

B+> 0x800065f0 <sum_then_double> jal ra,0x800065e2 <sum_to>
0x800065f4 <sum_then_double+4> li t0,2
0x800065f6 <sum_then_double+6> mul a0,a0,t0
0x800065fa <sum_then_double+10> ret
0x800065fc <sum_then_double+12> unimp
0x800065fe <sum_then_double+14> unimp
0x80006600 <sum_then_double+16> unimp
0x80006602 <sum_then_double+18> unimp
0x80006604 <sum_then_double+20> unimp
0x80006606 <sum_then_double+22> unimp
0x80006608 <sum_then_double+24> unimp
0x8000660a <sum_then_double+26> unimp

remote Thread 1.1 In: sum_then_double L?? PC: 0x800065f0
(gdb) c
Continuing.

Thread 1 hit Breakpoint 1, 0x00000000800065f0 in sum_then_double ()
=> 0x00000000800065f0 <sum_then_double+0>: ef f0 3f ff jal ra,0x80
0065e2 <sum_to>
(gdb) layout asm
(gdb) layout reg
(gdb) █
```

CSDN @Binary Oracle

我们可以看到现在的ra寄存器是0x80006392，它指向demo2函数，也就是sum_then_double的调用函数。之后我们执行代码，调用了sum_to:

```

Register group: general
zero      0x0      0
ra         0x800065f4    0x800065f4 <sum_then_double+4>
sp         0x3ffffff9f80 0x3ffffff9f80
gp         0x505050505050505 0x505050505050505
tp         0x0      0x0
t0         0x80002840    2147493952
t1         0x8000000000087fff -9223372036854218753
t2         0x505050505050505 361700864190383365
fp         0x3ffffff9f90 0x3ffffff9f90
s1         0x80012048    2147557448
a0         0x5      5
a1         0x3ffffff9f9c 274877882268

>0x800065e2 <sum_to> mv t0,a0
0x800065e4 <sum_to+2> li a0,0
0x800065e6 <loop> add a0,a0,t0
0x800065e8 <loop+2> addi t0,t0,-1
0x800065ea <loop+4> bnez t0,0x800065e6 <loop>
0x800065ee <loop+8> ret
B+ 0x800065f0 <sum_then_double> jal ra,0x800065e2 <sum_to>
0x800065f4 <sum_then_double+4> li t0,2
0x800065f6 <sum_then_double+6> mul a0,a0,t0
0x800065fa <sum_then_double+10> ret
0x800065fc <sum_then_double+12> unimp
0x800065fe <sum_then_double+14> unimp

remote Thread 1.1 In: sum_to L?? PC: 0x800065e2
(gdb) c
Continuing.

Thread 1 hit Breakpoint 1, 0x00000000800065f0 in sum_then_double ()
=> 0x00000000800065f0 <sum_then_double+0>: ef f0 3f ff jal ra,0x80
0065e2 <sum_to>
(gdb) layout asm
(gdb) layout reg
(gdb) si
0x00000000800065e2 in sum_to ()
=> 0x00000000800065e2 <sum_to+0>: aa 82 mv t0,a0 CSDN @Binary Oracle

```

我们可以看到ra寄存器的值被sum_to重写成了0x800065f4，指向sum_then_double，这也合理，符合我们的预期。我们在函数sum_then_double中调用了sum_to，那么sum_to就应该要返回到sum_then_double。

之后执行代码直到sum_then_double返回。因为没有恢复sum_then_double自己的Return address，现在的Return address仍然是sum_to对应的值，现在我们会进入到一个无限循环中。

接下来我们来看一些C代码。

demo4函数里面调用了dummysmain函数。我们在dummysmain函数中设置一个断点，

```
kernel/demos.c
22     {
23         printf("%d, %d, %d, %d, %d, %d, %d, %d, %d, %d\n", a, b, c,
24     }
25     void demo3()
26     {
27         _demo3('a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j');
28     }
29
30     int dummymain(int argc, char *argv[])
31     {
32         int i = 0;
B+>33     for (; i < argc; i++)
34     {
35         printf("Argument %d: %s\n", i, argv[i]);
36     }
37     return 0;
38 }
39
40 void demo4()
41 {
42     char *args[] = {"foo", "bar", "baz"};
43     int result = dummymain(sizeof(args)/sizeof(args[0]), args);
44     if (result < 0)
45         panic("Demo 4");
46 }
```

remote Thread 1.1 In: dummymain L33 PC: 0x8000642c
(gdb) c
Continuing.

Thread 1 hit Breakpoint 1, dummymain (argc=argc@entry=3, argv=argv@entry=0x3ffffff9f68) at kernel/demos.c:33
(gdb) █

CSDN @Binary Oracle

现在我们在dummymain函数中。如果我们在gdb中输入info frame，可以看到有关当前Stack Frame许多有用的信息。

```
(gdb) i frame
Stack level 0, frame at 0x3ffffff9f60:
 pc = 0x8000642c in dummymain (kernel/demos.c:33); saved pc = 0x800064b6
 called by frame at 0x3ffffff9f90
 source language c.
 Arglist at 0x3ffffff9f60, args: argc=argc@entry=3,
 argv=argv@entry=0x3ffffff9f68
 Locals at 0x3ffffff9f60, Previous frame's sp is 0x3ffffff9f60
 Could not fetch register "ustatus"; remote failure reply 'E14 CSDN @Binary Oracle
```

- Stack level 0，表明这是调用栈的最底层
- pc，当前的程序计数器
- saved pc，demo4的位置，表明当前函数要返回的位置

- source language c, 表明这是C代码
- Arglist at, 表明参数的起始地址。当前的参数都在寄存器中, 可以看到argc=3, argv是一个地址

如果输入backtrace (简写bt) 可以看到从当前调用栈开始的所有Stack Frame

```
(gdb) bt
#0  dummymain (argc=argc@entry=3, argv=argv@entry=0x3ffffff9f68)
    at kernel/demos.c:33
#1  0x00000000800064b6 in demo4 () at kernel/demos.c:43
#2  0x0000000080002f2c in sys_demo () at kernel/sysproc.c:149
#3  0x0000000080002bb6 in syscall () at kernel/syscall.c:174
#4  0x000000008000289e in usertrap () at kernel/trap.c:67
#5  0x000000000000001a in ?? ()
```

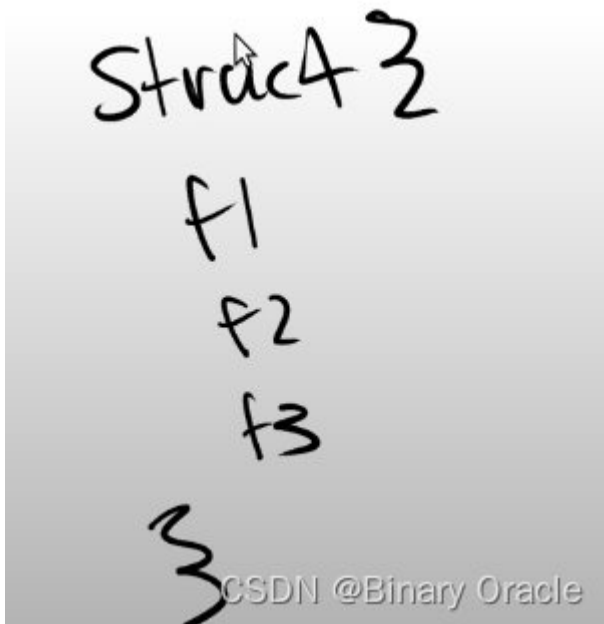
CSDN @Binary Oracle

如果对某一个Stack Frame感兴趣, 可以先定位到那个frame再输入info frame, Stack Frame中有更多的信息, 有一堆的Saved Registers, 有一些本地变量等等。这些信息对于调试代码来说超级重要。

Struct

struct在内存中的结构是怎样?

- 基本上来说, struct在内存中是一段连续的地址, 如果我们有一个struct, 并且有f1, f2, f3三个字段。
-



CSDN @Binary Oracle

当我们创建这样一个**struct**时，内存中相应的字段会彼此相邻。你可以认为**struct**像是一个数组，但是里面的不同字段的类型可以不一样。

我们可以将**struct**作为参数传递给函数。

```
struct Person {
    int id;
    int age;
    // char *name;
};

void printPerson(struct Person *p)
{
    printf("Person %d (%d)\n", p->id, p->age);
    // printf("Name:$s: %d (%d)\n", p->name, p->id, p->age);
}
```

CSDN @Binary Oracle

这里有一个名字是**Person**的**struct**，它有两个字段。我将这个**struct**作为参数传递给**printPerson**并打印相关的信息。我们在**printPerson**中设置一个断点，当程序运行到函数内部时打印当前的**Stack Frame**。

```
Thread 2 hit Breakpoint 5, printPerson (p=p@entry=0x3ffffff9f78)
  at kernel/demos.c:86
(gdb) i frame
Stack level 0, frame at 0x3ffffff9f70:
 pc = 0x80006594 in printPerson (kernel/demos.c:86); saved pc = 0x800065da
 called by frame at 0x3ffffff9f90
 source language c.
 Arglist at 0x3ffffff9f70, args: p=p@entry=0x3ffffff9f78
 Locals at 0x3ffffff9f70, Previous frame's sp is 0x3ffffff9f70
 Could not fetch register "ustatus"; remote failure reply 'E14'
```

CSDN @Binary Oracle

我们可以看到当前函数有一个参数**p**。打印**p**可以看到这是**struct Person**的指针，打印**p**的反引用可以看到**struct**的具体内容。

```
(gdb) p p
$4 = (struct Person *) 0x3ffffff9f78
(gdb) p *p
$5 = {id = 1215, age = 22}
```

CSDN @Binary Oracle

补充

函数调用约定

寄存器约定

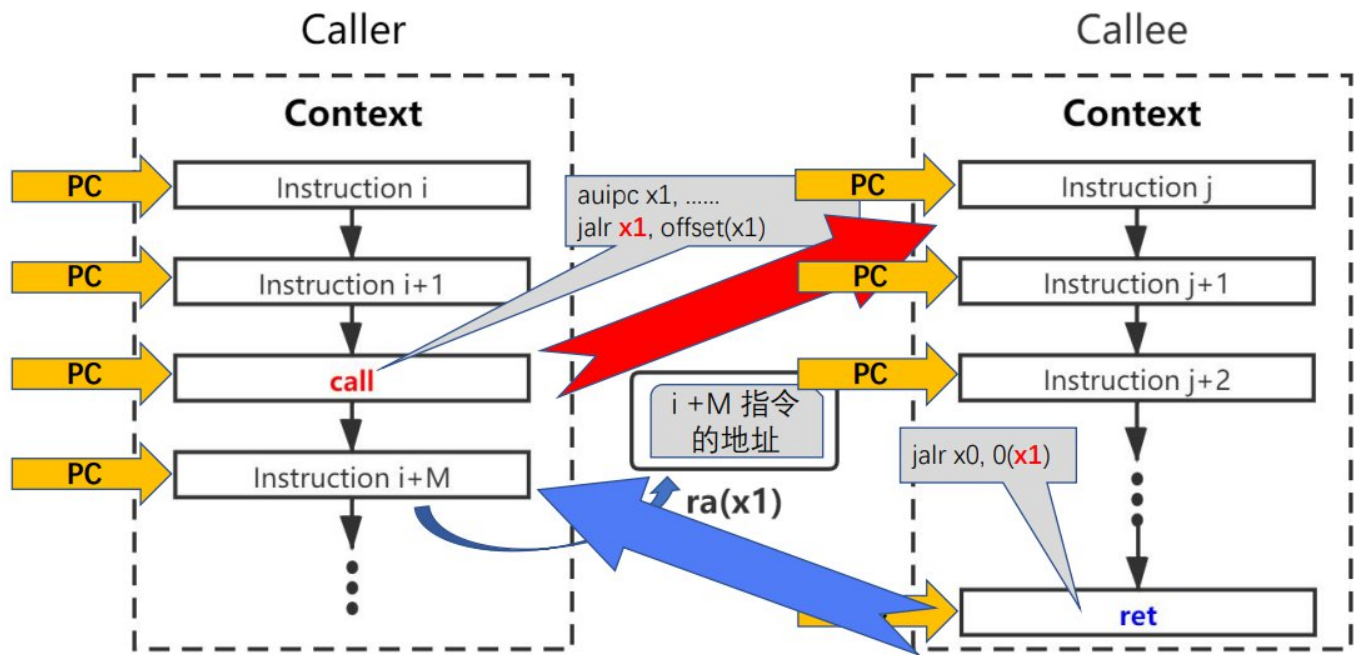
寄存器名	ABI 名（编程用名）	用途约定	谁负责在函数调用过程中维护这些寄存器
x0	zero	读取时总为 0，写入时不起任何效果	N/A
x1	ra	存放函数返回值（return address）	Caller
x2	sp	存放栈指针（stack pointer）	Callee
x5~x7, x28~x31	t0~t2, t3~t6	临时（temporaries）寄存器 ，Callee 可能会使用这些寄存器，所以 Callee 不保证这些寄存器中的值在函数调用过程中保持不变，这意味着对于 Caller 来说，如果需要的话，Caller 需要自己在调用 Callee 之前保存临时寄存器中的值。	Caller
x8, x9, x18~x27	s0, s1, s2~s11	保存（saved）寄存器 ，Callee 需要保证这些寄存器的值在函数返回后仍然维持函数调用之前的原值，所以一旦 Callee 在自己的函数中会用到这些寄存器则需要在栈中备份并在退出函数时进行恢复。	Callee
x10, x11	a0, a1	参数（argument）寄存器 ，用于在函数调用过程中保存第一个和第二个参数，以及在函数返回时传递返回值。	Caller
x12 ~ x17	a2 ~ a7	参数（argument）寄存器 ，如果函数调用时需要传递更多的参数，则可以用这些寄存器，但注意用于传递参数的寄存器最多只有 8 个（a0 ~ a7），如果还有更多的参数则要利用栈。	Caller

CSDN @Binary Oracle

函数跳转和返回指令的编程约定

伪指令	等价指令	描述	例子
jal offset	jal x1 , offset	跳转到 offset 制定位置，返回地址保存在 x1 (ra)	jal foo
jalr rs	jalr x1 , 0(rs)	跳转到 rs 中值所指定的位置，返回地址保存在 x1 (ra)	jalr s1
j offset	jal x0 , offset	跳转到 offset 制定位置， 不保存返回地址	j loop
jr rs	jalr x0 , 0(rs)	跳转到 rs 中值所指定的位置， 不保存返回地址	jr s1
call offset	auipc x1, offset[31 : 12] + offset[11] jalr x1 , offset[11:0](x1)	长跳转调用函数	call foo
tail offset	auipc x6, offset[31 : 12] + offset[11] jalr x0 , offset[11:0](x6)	长跳转 尾调用	tail foo
ret	jalr x0, 0(x1)	从 Callee 返回	ret

CSDN @Binary Oracle



CSDN @Binary Oracle

被调用函数的编程约定

```
def function ()
{
```

函数起始部分 (Prologue)

减少 sp 的值，根据本函数中使用 saved 寄存器的情况以及 local 变量的多少开辟栈空间。

将 saved 寄存器的值保存到栈中

如果函数中还会调用其他的函数，则将 ra 寄存器的值保存到栈中

函数执行体

函数退出部分 (Epilogue)

从栈中恢复 saved 寄存器

如果需要的话，从栈中恢复 ra 寄存器

增加 sp 的值，恢复到进入本函数之前的状态。

调用 ret 返回

```
}
```

Stack

函数的栈帧

CSDN @Binary Oracle

➤ 遵守 ABI (Abstract Binary Interface) 的规定

- 数据类型的大小，布局和对齐
- 函数调用约定 (Calling Convention)
- 系统调用约定
-

➤ RISC-V 函数调用约定规定:

- 函数参数采用寄存器 a0 ~ a7 传递
- 函数返回值采用寄存器 a0 和 a1 传递

CSDN @Binary Oracle

C 函数中嵌入 RISC-V 汇编

```
asm [volatile] (  
    “汇编指令”  
    : 输出操作数列表 (可选)  
    : 输入操作数列表 (可选)  
    : 可能影响的寄存器或者存储器 (可选)  
);
```

```
int foo(int a, int b)  
{  
    int c;  
    asm volatile (  
        "add %0, %1, %2"  
        : "=r"(c)  
        : "r"(a), "r"(b)  
    );  
    return c;  
}
```

- 汇编指令用双引号括起来，多条指令之间用 ";" 或者 "\n" 分隔
- “输出操作数列表” 和 “输入操作数列表” 用于将需要操作的 C 变量和汇编指令的操作数对应起来，多个操作数之间用 “,” 分隔。
- “可能影响的寄存器或者存储器” 用于告知编译器当前嵌入的汇编语句可能修改的寄存器或者内存，方便编译器执行优化。

更多见【参考 3】

CSDN @Binary Oracle

本文参与 腾讯云自媒体同步曝光计划，分享自作者个人站点/博客。